



this is cool. What's the difference between training this multi-head approach, with a contrastive approach with encoders pretraining to each other, to then having a encoder - model- decoder forecasting model, are these just different training stages? how would I design this pipeline?

Great question — you've basically spotted that there are two complementary design patterns here:

1. **Multi-head, joint training** (all encoders + dynamics + decoders trained together with reconstruction + dynamics + contrastive losses).
2. **Staged pipeline:**
 - Pretrain encoders to agree (contrastive across views / times),
 - Then train an encoder $\rightarrow$ dynamics $\rightarrow$ decoder forecasting model on top.

They're not contradictory; they're different points on a spectrum of **multi-stage self-supervised learning**. The literature on time-series SSL (contrastive pretraining $\rightarrow$ fine-tuning, CPC, TimeDART, TF-C, etc.) uses exactly this pattern: learn good representations first, then specialise for forecasting or other downstream tasks. ^{[1] [2] [3] [4] [5] [6]}

Below is how to think about the differences and how to design a clean pipeline.

1. Conceptual difference: joint vs staged

Joint multi-head training

You train everything end-to-end:

- Encoders $e^{(i)}$ for each view (SuperDARN, HF, etc.).
- Shared latent dynamics f .
- Decoders / observation heads $h^{(i)}$.
- Losses: reconstruction + dynamics + contrastive + regularisation.

Pros:

- All components co-adapt: the latent is shaped directly by what's needed for reconstruction and forecastability.

- No “representation gap” between pretraining and downstream use.
- Often best final performance if you have enough data and compute.

Cons:

- More complex optimisation (many moving parts).
- Harder to debug: if things go wrong, it’s not obvious whether it’s the encoders, dynamics, or decoders.
- Less modular: swapping in a new view or dynamics model means retraining everything.

Staged pipeline (pretrain encoders → train forecasting model)

You split training into phases:

1. Representation learning (pretraining):

Train encoders to produce a shared, discriminative latent space using mainly **contrastive** (and possibly reconstruction) losses, without worrying too much about dynamics.

2. Dynamics / forecasting training:

Freeze or fine-tune the encoders, and train:

- A dynamics model f in latent space.
- Optionally, decoders / task heads (forecasting specific observables, indices, etc.).

This is exactly what methods like TF-C, TimeDART, and many **contrastive predictive coding (CPC)** variants do: self-supervised pretraining on unlabeled data, then fine-tuning for a downstream task (forecasting, classification, etc.). ^{[2] [3] [4] [5] [6] [1]}

Pros:

- Cleaner separation of concerns:
 - Stage 1: “What is a good representation?”
 - Stage 2: “How does this representation evolve and how do we forecast?”
- Easier to experiment:
 - Try different dynamics models without retraining encoders.
 - Plug in new views by fine-tuning only their encoders.
- Matches common SSL practice in time series, where **fine-tuning end-to-end** after contrastive pretraining often beats using a frozen encoder. ^{[4] [7]}

Cons:

- Risk of a mismatch between pretraining objective and downstream needs if stages are too decoupled.
- May need a final **joint fine-tuning** stage to fully align everything.

In practice, the best results often come from a **hybrid**:

- Pretrain encoders with contrastive + reconstruction.

- Train dynamics + decoders.
- Then do a short **end-to-end fine-tuning** of everything together.

2. A concrete pipeline design for your ionosphere project

Here's a pragmatic, three-stage pipeline that matches what you described and aligns with SSL best practices. [\[3\]](#) [\[1\]](#) [\[2\]](#) [\[4\]](#)

Stage 1: Multi-view contrastive encoder pretraining

Goal: Learn a shared latent space where:

- Different views (SuperDARN, HF, indices) at the same time are close.
- Different times are separable.
- The latent is expressive and well-structured.

Architecture:

- Per-view encoders:
 - e_{SD} : SuperDARN $\rightarrow \mathbf{z}$
 - e_{HF} : HF connectivity/height $\rightarrow \mathbf{z}$
 - optionally e_{idx} : other indices $\rightarrow \mathbf{z}$
- No dynamics model yet; just encoders and maybe simple decoders.

Losses:

1. Contrastive loss (as we designed):

- Positives: different views at the same time.
- Negatives: different times.

2. Optional reconstruction loss:

- Add simple decoders $d^{(i)}$ for each view:

$$\hat{\text{matbfy}}^{(i)} = d^{(i)}(e^{(i)}(\text{matbfy}^{(i)}))$$
- MSE / BCE per view.
- This keeps the latent grounded in the actual observation space.

Total:

$$\mathcal{L}_{\text{stage1}} = \mathcal{L}_{\text{cont}} + \lambda_{\text{recon}} \sum_i \mathcal{L}_{\text{recon}}^{(i)}$$

Output:

- Trained encoders $e^{(i)}$ that map each view into a shared latent space.
- Optionally, decoder weights you can reuse later.

Practical notes:

- You can train this on long stretches of historical data with no labels.
- Use fairly large batches (many timesteps) to get good contrastive signal.
- This stage is where you “fill the expressability range” of the latent.

Stage 2: Latent dynamics + observation heads (forecasting model)

Goal: Learn how the latent evolves and how to map it back to observables.

Architecture:

- Freeze or partially freeze encoders $e^{(i)}$.
- Define a shared latent per timestep, e.g.:

$$\mathbf{z}_t = \frac{1}{V} \sum_i e^{(i)}(\mathbf{y}_t)$$

- Train:
 - Dynamics model $f_\theta: \mathbf{z}_{t+1} = f_\theta(\mathbf{z}_t, \mathbf{u}_t)$.
 - Observation heads (decoders) $h^{(i)}$:
 - SuperDARN head: $\mathbf{z} \rightarrow$ predicted LOS velocity / power.
 - HF head: $\mathbf{z} \rightarrow$ link probability + bounce height.
 - Optional index head.

Losses:

1. Reconstruction / observation loss:

$$\mathcal{L}_{\text{recon}} = \sum_i w_i \mathbb{E}_{t \sim \mathcal{D}} [\|\mathbf{y}_t^{(i)} - h^{(i)}(\mathbf{z}_t)\|^2]$$

2. Dynamics / prediction loss:

]

3. Optional contrastive regularisation:

- Keep a small contrastive term to maintain latent structure:

Total:

$$\mathcal{L}_{\text{stage2}} = \mathcal{L}_{\text{recon}} + \lambda_{\text{dyn}} \mathcal{L}_{\text{dyn}} + \lambda_{\text{cont}} \mathcal{L}_{\text{cont}}^{\text{stage2}}$$

Training options:

- **Frozen encoders:**
 - Keep $e^{(i)}$ fixed; only train f_θ and $h^{(i)}$.
 - Good for isolating dynamics learning.
- **Light fine-tuning:**
 - Allow a small learning rate on encoders so they can adapt to the forecasting objective.
 - Often yields better performance than fully frozen encoders. ^[4]

Output:

- A latent dynamics model that can forecast $\mathbf{z}_{t+k}$ from $\mathbf{z}_t$ and OMNI drivers.
- Observation heads that map latent states to each project's measurement space.

Stage 3: End-to-end fine-tuning (optional but recommended)

Goal: Remove any residual mismatch between representation and forecasting objectives.

Architecture:

- Unfreeze everything:
 - Encoders $e^{(i)}$
 - Dynamics f_θ
 - Decoders / heads $h^{(i)}$
- Use the same loss as in Stage 2 (reconstruction + dynamics + optional contrastive), but now **end-to-end**.

This is exactly what time-series SSL papers find: **fine-tuning the whole model** (encoder + task head) usually beats keeping the encoder frozen, even if it was well pretrained. ^[7] ^[4]

Loss:

Same as Stage 2, possibly with:

- Lower learning rate.
- Smaller λ_{cont} (the latent is already well-structured).

3. How this maps to “multi-head joint training” vs “encoder–model–decoder”

Your two descriptions map neatly onto this pipeline:

- **“Multi-head approach with contrastive encoders pretraining to each other”**
→ Primarily Stage 1 (multi-view contrastive pretraining), possibly with some reconstruction.
- **“Encoder–model–decoder forecasting model”**
→ Stage 2 (latent dynamics + observation heads) and Stage 3 (end-to-end fine-tuning).

So yes: these are best viewed as **different training stages** in a single coherent pipeline, not competing architectures.

You could also do a simpler variant:

- Skip explicit Stage 1.
- Train everything jointly from scratch with reconstruction + dynamics + contrastive (the “joint multi-head” setup we wrote earlier).
- Then optionally fine-tune with a stronger focus on forecastability.

That's valid too, but staging usually gives:

- Better initial representations.
- Easier debugging.
- More flexibility to reuse encoders or dynamics models independently.

4. Suggested concrete pipeline for your case

Given your three projects (SuperDARN, OMNI+ML, HF network), a practical design:

Data & interfaces

- Define:
 - `y_sd[t]`: SuperDARN fitacf-derived features per time.
 - `y_hf_link[t]`, `y_hf_height[t]`: HF connectivity + bounce height.
 - `u[t]`: OMNI features (solar wind, IMF, coupling functions).
- Fix:
 - Latent dimension (e.g., 32–128).
 - Grid or abstract latent (you can start abstract, then later interpret as fields).

Stage 1: Multi-view contrastive encoder pretraining

- Encoders:
 - `enc_sd: y_sd → z`
 - `enc_hf: cat(link, height) → z`
 - optionally `enc_idx`.
- Loss:
 - Contrastive across time, agreement across views.
 - Optional simple decoders to reconstruct each view (keeps latents grounded).

Train until:

- Different views at the same time cluster tightly in latent space.
- Different times are well separated.

Stage 2: Latent dynamics + observation heads

- Shared latent:
 - `z[t] = mean(enc_sd(y_sd[t]), enc_hf(...), ...)`
- Dynamics:
 - `z_next_pred = f(z[t], u[t])`
- Heads:
 - `head_sd(z) → y_sd_pred`

- `head_hf(z) → (link_pred, height_pred)`

- Loss:
 - Reconstruction for each head.
 - Dynamics prediction loss.
 - Light contrastive regularisation.

Train until:

- Good short-term forecast skill in latent space.
- Reasonable reconstruction of all views from latent.

Stage 3: End-to-end fine-tuning

- Unfreeze encoders + dynamics + heads.
- Train with same losses, lower LR.
- Optionally emphasise longer-horizon forecasting metrics (e.g., 1–4 hour ahead).

5. When to use which variant

- If you have lots of **unlabeled data** and want maximum flexibility:
 - Use the full 3-stage pipeline.
- If you have **limited data** or want a simpler setup:
 - Train jointly from scratch with reconstruction + dynamics + contrastive, then fine-tune.
- If you plan to **add/remove views** (new radars, new HF stations, new indices):
 - Staged is nicer:
 - Pretrain new encoder in Stage 1.
 - Plug into Stage 2/3 with minimal retraining.

If you tell me your preferred latent size, batch length (how many timesteps you want in a batch), and which views you definitely have ready (e.g., “SuperDARN + HF, no indices yet”), I can sketch a concrete config (modules, losses, learning rates, and stage durations) tailored to your setup.

✱✱

1. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11199859/>
2. <https://arxiv.org/html/2410.05711v4>
3. <https://zitniklab.hms.harvard.edu/projects/TF-C/>
4. <https://rbcborealis.com/research-blogs/what-constitutes-good-contrastive-learning-in-time-series-forecasting/>
5. <https://openreview.net/pdf?id=OJ4mMfGKLN>
6. <https://www.alphaxiv.org/abs/2410.05711>
7. <https://openreview.net/pdf?id=yGv5GzIBwr>

8. <https://www.sciencedirect.com/science/article/pii/S0020025526010212>
9. <https://github.com/qingsongedu/Awesome-SSL4TS>
10. <https://www.activeloop.ai/resources/glossary/contrastive-predictive-coding-cpc/>
11. <https://www.alphaxiv.org/abs/1807.03748>
12. <https://www.lesswrong.com/posts/XE6LD2c9NtB7gMdEm/an-92-learning-good-representations-with-contrastive>
13. <https://www.ibm.com/think/topics/self-supervised-learning>
14. <https://arxiv.org/abs/1807.03748>
15. <https://cea.hal.science/cea-03547497/document>